# Sorting Algorithms

ณัฐนันท์ จารุกรกุล 66070501017
ธิติ ติยะชัยพานิช 66070501026
รัชชนนท์ ทิพพามูล 66070501045
ธรรมภณ แฉ่งพาที 66070501076

Department of Computer Engineering
King Mongkut's University of Technology Thonburi

## I. บทนำ

### A. ความหมายของ Sorting Algorithm

Sorting Algorithm หรือ ขั้นตอนวิธีการเรียงลำดับ คือ กระบวนการจัดเรียงข้อมูลให้อยู่ในลำดับที่ต้องการ ไม่ว่าจะเป็นการเรียงจากน้อยไปมาก (ascending order) หรือจากมากไปน้อย (descending order) โดยอัลกอริทึมการเรียงลำดับแต่ละประเภทจะมีวิธีการทำงานที่แตกต่างกันออกไป เช่น การเปรียบเทียบและสลับตำแหน่ง (comparison-based) หรือการใช้การนับและการกระจาย (non-comparison based)

ในปัจจุบันมี Sorting Algorithm หลากหลายรูปแบบที่ถูกพัฒนาขึ้นเพื่อตอบสนองความต้องการที่แตกต่างกัน แต่ละอัลกอริทึมจะมีประสิทธิภาพและข้อจำกัดที่แตกต่างกัน เช่น Selection Sort และ Bubble Sort ที่มีการทำงานแบบง่าย แต่ใช้เวลามาก ในขณะที่ Quick Sort และ Merge Sort มีประสิทธิภาพสูงกว่า แต่มีความซับซ้อนมากกว่า การเลือกใช้ Sorting Algorithm จึงขึ้นอยู่กับปัจจัยต่างๆ เช่น ขนาดของข้อมูล รูปแบบข้อมูล และทรัพยากรที่มี

### B. เป้าหมายการปรับปรุง Sorting Algorithm

การปรับปรุง Sorting Algorithm มีเป้าหมายหลักคือการเพิ่มประสิทธิภาพในการทำงานของอัลกอริทึม โดยพิจารณาจากหลายปัจจัย ได้แก่ Time Complexity ที่ต้องการลดระยะเวลาการทำงานให้น้อยที่สุด ทั้งใน worst case, average case และ best case, Space Complexity ที่ต้องการใช้หน่วยความจำเพิ่มเติมให้น้อยที่สุด และความเสถียรของอัลกอริทึม (Stability) ที่ต้องรักษาลำดับของข้อมูลที่มีค่าเท่ากันไว้ตามลำดับเดิม

นอกจากนี้ การปรับปรุง Sorting Algorithm ยังมุ่งเน้นให้สามารถรองรับข้อมูลที่หลากหลายและมีความยืดหยุ่นในการใช้งาน เช่น การพัฒนา Distribution-Counting Sort ที่เหมาะกับข้อมูลที่มีช่วงจำกัด หรือ Comparison-Counting Sort ที่สามารถใช้กับข้อมูลที่เปรียบเทียบกันได้ทั่วไป รวมถึงการพัฒนา Heap Sort ที่ต้องการใช้พื้นที่หน่วยความจำเพิ่มเติมน้อย ทั้งนี้เพื่อให้สามารถเลือกใช้อัลกอริทึมที่เหมาะสมกับลักษณะข้อมูลและทรัพยากรที่มีอยู่มากที่สุด

### C. จุดประสงค์การทดลอง

การทดลองนี้มีจุดประสงค์เพื่อศึกษาและเปรียบเทียบประสิทธิภาพของอัลกอริทึมการเรียงลำดับทั้ง 8 ประเภท ได้แก่ Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Comparison-Counting Sort และ Distribution-Counting Sort โดยมุ่งเน้นการวิเคราะห์ Time Complexity ในกรณีที่ดีที่สุด (Best Case) และแย่ที่สุด (Worst Case) ของแต่ละอัลกอริทึม รวมถึงการวัดประสิทธิภาพการทำงานจริงในรูปแบบของเวลาที่ใช้ในการประมวลผล

นอกจากนี้ การทดลองยังมีเป้าหมายในการระบุอัลกอริทึมที่มีประสิทธิภาพสูงสุดสำหรับสถานการณ์การใช้งานที่แตกต่างกัน โดยจะทำการทดสอบกับข้อมูลนำเข้าที่มีลักษณะและขนาดที่หลากหลาย เพื่อให้สามารถให้คำแนะนำในการเลือกใช้อัลกอริทึมที่เหมาะสมที่สุดสำหรับแต่ละสถานการณ์ รวมถึงวิเคราะห์ข้อดีและข้อจำกัดของแต่ละอัลกอริทึมในการจัดการกับข้อมูลรูปแบบต่าง ๆ

## II. ทบทวนวรรณกรรม

### A. Selection Sort

เป็นอัลกอริทึมการเรียงลำดับที่มีความเรียบง่าย แนวคิดหลักคือ "การค้นหา" องค์ประกอบที่น้อยที่สุด หรือ มากที่สุด จากเซตข้อมูล และ "การแลกเปลี่ยน" ให้มาอยู่ในตำแหน่งที่ถูกต้อง เป็นการทำซ้ำ ๆ ไปจนกระทั่งข้อมูลทั้งหมดถูกเรียงลำดับเรียบร้อย

**Algorithm 1** Selection Sort

```
Input: An array A[0...n − 1] of orderable elements
Output: Array A[0...n − 1] sorted in nondecreasing order
for i ← 0 to n − 2 do
    min ← i
    for j ← i + 1 to n − 1 do
        if A[j] < A[min] then
            min ← j
        end if
    end for
    swap A[i] and A[min]
end for
```

Selection Sort มี Time Complexity $O(n^2)$ ใน worst-case, average-case, และ best-case โดยใช้ในการแก้ปัญหา Simple Array Sorting หรือปัญหาที่มี memory space จำกัด [1] [2] [3]

B. Bubble Sort

เป็นอัลกอริทึมการจัดเรียง ที่จะมีการเปรียบเทียบข้อมูลสองตำแหน่งที่อยู่ติดกัน ถ้าข้อมูลเรียงกันไม่ถูกต้องก็จะถูกสลับไปเรื่อย ๆ จนถึงคู่สุดท้าย กระบวนการนี้จะดำเนินไปเรื่อย ๆ จนกว่าชุดข้อมูลจะถูกเรียงทั้งหมด โดยจะเรียงจากน้อยไปมาก (Ascending) หรือมากไปน้อย (Descending) ก็ได้

**Algorithm 2** Bubble Sort

```
Input: An array A[0...n − 1] of orderable elements
Output: Array A[0...n − 1] sorted in nondecreasing order
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j + 1] < A[j] then
            swap A[j] and A[j + 1]
        end if
    end for
end for
```

Bubble Sort มี Time Complexity $O(n^2)$ ใน worst-case และ average-case โดยสำหรับ best-case เป็น $O(n)$ โดยใช้ในการแก้ปัญหา Simple Array Sorting หรือ Nearly-Sorted Array [1] [4] [5]

C. Insertion Sort

การจัดเรียงแบบแทรก เป็นกระบวนการการจัดเรียงอย่างง่าย มีหลักการคล้ายกับการเรียงไพ่ในมือโดยชุดข้อมูลถูกแบ่งออกเป็นสองส่วน คือ ส่วนที่จัดเรียงแล้วกับส่วนที่ยังไม่จัดเรียง โดยเมื่อเริ่มกระบวนการจะตรวจสอบข้อมูลในส่วนที่ยังไม่ถูกจัดเรียงแล้วปรียบเทียบว่าควรนำไปแทรกในตำแหน่งใดของส่วนที่จัดเรียงแล้ว ทำไปเรื่อย ๆ จนไม่เหลือส่วนที่ยังไม่ถูกจัดเรียง

**Algorithm 3** Insertion Sort

```
Input: An array A[0...n − 1] of orderable elements
Output: Array A[0...n − 1] sorted in nondecreasing order
for i ← 1 to n − 1 do
    v ← A[i]
    j ← i − 1
    while j ≥ 0 and A[j] > v do
        A[j + 1] ← A[j]
        j ← j − 1
    end while
    A[j + 1] ← v
end for
```

Selection Sort มี Time Complexity $O(n^2)$ ใน worst-case และ average-case ที่ $O(\frac{n^2}{4})$ โดยสำหรับ best-case เป็น $O(n^2)$ โดยใช้ในการแก้ปัญหา Simple Array Sorting และ Card Sorting [1]

D. Merge Sort

เป็น อัล กอ ริ ทึม แบบ แบ่ง แยก และ เอาชนะ (divide-and-conquer) มีการแบ่งแยก โดยแบ่งข้อมูลออกเป็นสองส่วนจากนั้นให้ recursive เรียกตัวเองเรื่อย ๆ จนข้อมูลนั้นถูกแยกส่วนจนเล็กที่สุด ต่อมาเอาชนะเป็นการเอาแต่ละส่วนมาพิจารณารวมกันจนได้เป็นส่วนใหญ่ที่เรียงลำดับเรียบร้อยแล้ว โดยส่วนที่เอามาพิจารณาต้องมีการเรียงลำดับใหม่ทุกครั้ง

**Algorithm 4** Merge Sort

Input: An array $A[0...n-1]$ of orderable elements
Output: Array $A[0...n-1]$ sorted in nondecreasing order
if $n > 1$ then
  copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$
  copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lfloor n/2 \rfloor - 1]$
  MergeSort($B[0..\lfloor n/2 \rfloor - 1]$)
  MergeSort($C[0..\lfloor n/2 \rfloor - 1]$)
  Merge($B, C, A$)
end if

Merge Sort มี Time Complexity $O(n \log n)$ ใน worst-case, best-case และ average-case โดยใช้ในการแก้ปัญหา Array Sorting และเหมาะสำหรับชุดข้อมูลขนาดใหญ่ที่ต้องการความเสถียรและประสิทธิภาพที่แน่นอน [1] [6] [7]

## E. Quick Sort

เป็นวิธีการเรียงลำดับแบบหนึ่ง ที่ทำได้ด้วยวิธีการแบ่งแยกและเอาชนะ (divide-and-conquer) และมีลักษณะการทำงานแบบเวียนเกิด โดยการแบ่งข้อมูลในกลุ่ม กลุ่มหนึ่งออกเป็นสองกลุ่มย่อยที่แยกการทำงานจากกันอย่างอิสระ เริ่มจากการเลือกข้อมูลจากกลุ่มข้อมูลขึ้นมาหนึ่งค่า โดยจะเรียกข้อมูลตัวนี้ว่าข้อมูลเปรียบเทียบ หรือ Pivot หลังจากเลือกค่าแล้ว จะใช้ Pivot ที่เลือกมาเป็นค่าหลักในการแบ่งข้อมูลออกเป็นสองส่วน คือ ส่วนที่อยู่ตอนหน้า หรือกลุ่มทางซ้าย (ส่วนนี้จะมีค่าน้อยกว่าหรือเท่ากับข้อมูล Pivot) ส่วนที่อยู่ตรงกลาง (คือค่า Pivot) และส่วนที่อยู่ตอนหลัง หรือกลุ่มทางขวา (มีค่ามากกว่าข้อมูล Pivot) หลังจากแบ่งส่วนแล้ว จะมีการนำแต่ละส่วนไปแบ่งย่อยด้วยวิธีการเดียวกับข้างต้นอีก และทำจนกระทั่งข้อมูลทุกส่วนเหลือเพียงค่าเดียวไม่สามารถแบ่งข้อมูลย่อยได้อีก

**Algorithm 5** Quick Sort

Input: Subarray of array $A[0...n-1]$, defined by its left and right indices $l$ and $r$
Output: Subarray $A[l...r]$ sorted in nondecreasing order
if $l < r$ then
  $s \leftarrow$ Partition($A[l...r]$)  ▷ $s$ is a split position
  QuickSort($A[l...s-1]$)
  QuickSort($A[s+1...r]$)
end if

Quick Sort มี Time Complexity $O(n \log n)$ ใน best-case และ average-case แต่ใน worst-case เป็น $O(n^2)$ โดยใช้ในการแก้ปัญหา General-Purpose Sorting โดยเฉพาะสำหรับ array ขนาดใหญ่ที่ต้องการประสิทธิภาพเฉลี่ยที่ดี [1] [8]

## F. Heap Sort

อัลกอริทึมการเรียงลำดับข้อมูลแบบ Heap Sort ได้มีการนำวิธีการเรียงลำดับข้อมูลแบบ Selection Sort มาปรับปรุงให้มีประสิทธิภาพดียิ่งขึ้นโดย Heap Sort จะเริ่มต้นทำงานจากอาร์เรย์ที่มีการจัดเรียงลำดับข้อมูลแบบฮีพ ซึ่งคุณสมบัติสำคัญของฮีพทรีก็คือ รูทโหนดจะมีค่ามากที่สุด และในการแทนฮีพทรีในอาร์เรย์ค่าที่มากที่สุดจะอยู่ที่ตำแหน่งแรก ดังนั้นเราจะนำค่าที่มากที่สุดในตำแหน่งแรกสลับกับค่าสุดท้ายของฮีพและถือว่าโหนดสุดท้ายนี้ได้หลุดออกจากฮีพทรีแล้ว จากนั้นสมาชิกหรือข้อมูลส่วนที่เหลือก็จะมีการรีฮีพโครงสร้างใหม่ ซึ่งค่าที่มากที่สุดก็จะอยู่ที่ตำแหน่งแรกเช่นเดิมและจะดำเนินการเช่นนี้ไปจนกระทั่งได้จัดการกับข้อมูลทั้งหมดในฮีพ ก็จะได้ข้อมูลที่จัดเรียงเรียบร้อยอย่างสมบูรณ์

**Algorithm 6** Heap Sort

Input: An array $A[0...n-1]$ of orderable elements
Output: Array $A[0...n-1]$ sorted in nondecreasing order
Build_Max_Heap($A$)
for $i \leftarrow$ length$[A]$ downto $2$ do
  exchange $A[1]$ and $A[i]$
  heap-size$[A] \leftarrow$ heap-size$[A] - 1$
  Max_Heapify($A, 1$)
end for

Heap Sort มี Time Complexity $O(n \log n)$ ใน worst-case, best-case และ average-case โดยใช้ในการแก้ปัญหาการเรียงลำดับที่ต้องการพื้นที่หน่วยความจำเพิ่มเติม $O(1)$ และไม่จำเป็นต้องกังวลกับการคงความเสถียร [1] [9] [10]

### G. Comparison-Counting Sort

อัลกอริทึมนี้ใช้หลักการนับจำนวนการเปรียบเทียบระหว่างสมาชิกแต่ละตัวเพื่อระบุตำแหน่งของแต่ละตัวในลำดับที่เรียง แล้วการนับการเปรียบเทียบสำหรับสมาชิกแต่ละตัวในลิสต์ให้เปรียบเทียบกับสมาชิกอื่น ๆ ทั้งหมด จากนั้นนับจำนวนสมาชิกที่มีค่าน้อยกว่าสมาชิกปัจจุบันและเก็บผลลัพธ์นี้ไว้

ผลการนับจะบอกตำแหน่งที่สมาชิกนั้นควรอยู่ในลำดับที่เรียงแล้ว โดยถ้ามีจำนวนการเปรียบเทียบที่สมาชิกนั้นชนะได้ k ครั้ง ตำแหน่งของมันควรจะเป็นตำแหน่งที่ k+1 สุดท้ายจึงวางสมาชิกแต่ละตัวลงในตำแหน่งที่กำหนดตามการนับ

**Algorithm 7** Comparison-Counting Sort

```
Input: An array A[0...n−1] of orderable elements
Output: Array S[0...n−1] of A's elements sorted in nondecreasing order
for i ← 0 to n−1 do
    Count[i] ← 0
end for
for i ← 0 to n−2 do
    for j ← i+1 to n−1 do
        if A[i] < A[j] then
            Count[j] ← Count[j] + 1
        else
            Count[i] ← Count[i] + 1
        end if
    end for
end for
for i ← 0 to n−1 do
    S[Count[i]] ← A[i]
end for
```

Comparison Counting Sort มี Time Complexity $O(n^2)$ ใน worst-case, best-case และ average-case โดยใช้ในการแก้ปัญหาการเรียงลำดับข้อมูลที่มีขนาดเล็กและต้องการความเสถียรในการเรียง [1]

### H. Distribution-Counting Sort

เป็นอัลกอริทึมการเรียงลำดับที่ไม่อาศัยการเปรียบเทียบโดยตรงระหว่างค่าในลิสต์ และมีหลักการที่คล้ายคลึงกับ Counting Sort โดยจะใช้การนับจำนวนครั้งที่ตัวเลขหรือค่าปรากฏในลิสต์ จากนั้นใช้ข้อมูลเหล่านั้นในการกระจายตำแหน่งที่เหมาะสมของแต่ละตัวในลิสต์ที่เรียงลำดับแล้ว

**Algorithm 8** Distribution-Counting Sort

```
Input: An array A[0...n−1] of integers between l and u (l ≤ u)
Output: Array S[0...n−1] of A's elements sorted in nondecreasing order
for j ← 0 to u−l do
    D[j] ← 0
end for
for i ← 0 to n−1 do
    D[A[i]−l] ← D[A[i]−l] + 1
end for
for j ← 1 to u−l do
    D[j] ← D[j−1] + D[j]
end for
for i ← n−1 downto 0 do
    j ← A[i]−l
    S[D[j]−1] ← A[i]
    D[j] ← D[j]−1
end for
```

Distribution Counting Sort มี Time Complexity $O(n+k)$ ใน worst-case, best-case และ average-case โดยที่ $k$ คือช่วงของข้อมูลนำเข้า โดยใช้ในการแก้ปัญหาการเรียงลำดับข้อมูลที่เป็นจำนวนเต็มหรือข้อมูลที่มีประเภทจำกัดในช่วงที่ทราบค่า [1]

## III. ระเบียบวิธี

### A. การออกแบบการทดลอง

1) ขนาดข้อมูลนำเข้า: การทดลองจะใช้ขนาดข้อมูลนำเข้าที่แตกต่างกัน 4 ขนาด ได้แก่ 1,000, 5,000, 10,000, และ 25,000 ตัว เพื่อศึกษาผลกระทบของขนาดข้อมูลต่อประสิทธิภาพของแต่ละอัลกอริทึมให้เห็นความแตกต่างของประสิทธิภาพในการทำงานได้ชัดเจน โดยกำหนดให้ค่าของข้อมูลอยู่ในช่วง 0 ถึง 999,999

2) รูปแบบข้อมูล: การทดลองจะทำการทดสอบกับข้อมูล 3 รูปแบบ เพื่อวิเคราะห์ประสิทธิภาพของอัลกอริทึมในสถานการณ์ต่างๆ:

- Random Order: ข้อมูลที่มีการสุ่มค่าแบบสมบูรณ์
- Ascending Order: ข้อมูลที่เรียงจากน้อยไปมาก
- Descending Order: ข้อมูลที่เรียงจากมากไปน้อย

3) จำนวนครั้งในการทดลอง: แต่ละการทดสอบจะทำการทดลองซ้ำ 5 ครั้ง เพื่อหาค่าเฉลี่ยของเวลาที่ใช้ในการประมวลผล ทั้งนี้เพื่อลดผลกระทบจากความแปรปรวนของระบบและกระบวนการทำงานอื่น ๆ ของคอมพิวเตอร์ โดยในแต่ละครั้งของการทดลองจะมีการสร้างชุดข้อมูลใหม่ทั้งหมด ยกเว้นกรณี Ascending และ Descending Order ที่จะใช้ชุดข้อมูลเดิม

การทดลองจะดำเนินการทั้งหมด 480 ครั้ง (8 อัลกอริทึม × 4 ขนาดข้อมูล × 3 รูปแบบข้อมูล × 5 ครั้งต่อการทดสอบ)

4) วิธีประเมินผลการทดลอง: การประเมินผลจะพิจารณาจากเวลาเฉลี่ยในการประมวลผล (Average Execution Time) วัดโดยใช้ std::chrono::high_resolution_clock โดยผลการทดลองจะถูกบันทึกในรูปแบบ CSV file และนำมาวิเคราะห์โดยใช้กราฟแสดงความสัมพันธ์ระหว่างขนาดข้อมูลและเวลาที่ใช้ในการประมวลผล รวมถึงการเปรียบเทียบประสิทธิภาพระหว่างอัลกอริทึมในแต่ละรูปแบบข้อมูล

5) ตัวแปรต้น ตัวแปรตาม ตัวแปรควบคุม: ตัวแปรต้น:

- ประเภทของอัลกอริทึมการเรียงลำดับ (8 อัลกอริทึม)
- ขนาดของข้อมูลนำเข้า (4 ขนาด)
- รูปแบบการจัดเรียงของข้อมูลนำเข้า (3 รูปแบบ)

ตัวแปรตาม:

- เวลาที่ใช้ในการประมวลผล (milliseconds)
- ความถูกต้องของการเรียงลำดับ

ตัวแปรควบคุม:

- สภาพแวดล้อมการทดสอบ (Hardware และ Software)
  - CPU: Intel Core i5 10th GEN
  - RAM: 24GB
  - OS: Windows 11
  - Compiler: g++ 6.3.0
  - IDE: Visual Studio Code
- กระบวนการทำงานเบื้องหลังของระบบ
- อุณหภูมิห้องและสภาพแวดล้อมการทดสอบ

## IV. ผลการทดลอง

### A. ชุดข้อมูล 1,000 ตัว

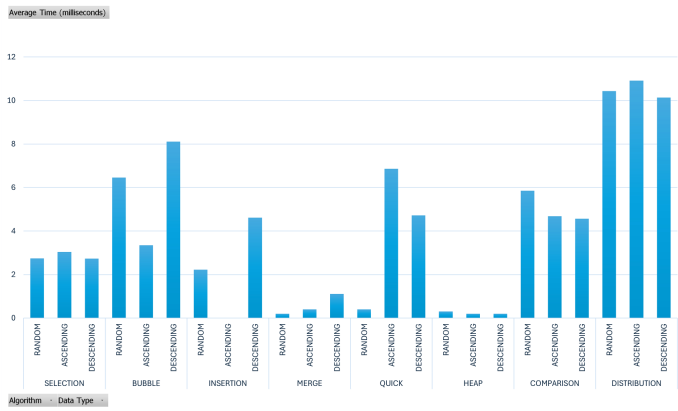


Fig. 1: chart showing results for randomized, ascending, and descending average execution time for all algorithms for 1,000 input data points

สำหรับชุดข้อมูลขนาด 1,000 ตัว ผลการทดสอบจาก (1) พบว่า Merge Sort และ Heap Sort มีประสิทธิภาพดีที่สุด โดยใช้เวลาเฉลี่ยน้อยกว่า 1 มิลลิวินาที ในการเรียงข้อมูลทุกรูปแบบ (สุ่ม, เรียงจากน้อยไปมาก, เรียงจากมากไปน้อย) Insertion Sort มีประสิทธิภาพดีมากในกรณีข้อมูลเรียงจากน้อยไปมากอยู่แล้ว แต่ประสิทธิภาพจะลดลงอย่างมากเมื่อข้อมูลเรียงจากมากไปน้อย

Bubble Sort และ Distribution-Counting Sort เป็นอัลกอริทึมที่ใช้เวลามากที่สุด โดย Bubble Sort ใช้เวลาประมาณ 6-8 มิลลิวินาทีสำหรับข้อมูลสุ่มและข้อมูลเรียงจากมากไปน้อย ส่วน Distribution Sort ใช้เวลาค่อนข้างคงที่ประมาณ 10 มิลลิวินาที

Quick Sort แสดงให้เห็นพฤติกรรมที่น่าสนใจ โดยทำงานได้ดีกับข้อมูลสุ่ม (0.4 มิลลิวินาที) แต่ประสิทธิภาพลดลงเมื่อข้อมูลเรียงลำดับอยู่แล้ว โดยใช้เวลา 4-6 มิลลิวินาทีสำหรับข้อมูลที่เรียงลำดับทั้งจากน้อยไปมาก และมากไปน้อย

B. ชุดข้อมูล 5,000 ตัว

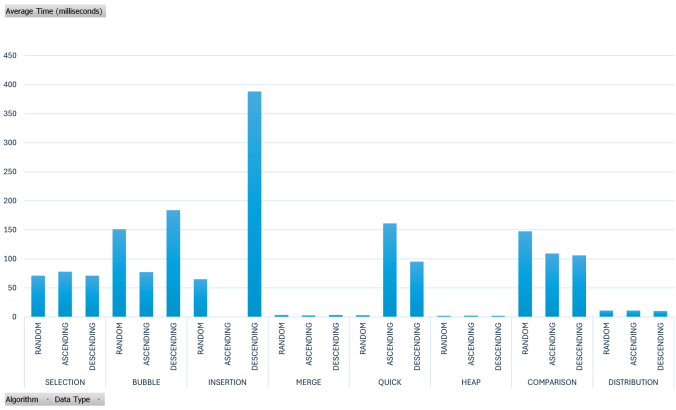


Fig. 2: chart showing results for randomized, ascending, and descending average execution time for all algorithms for 5,000 input data points

C. ชุดข้อมูล 10,000 ตัว

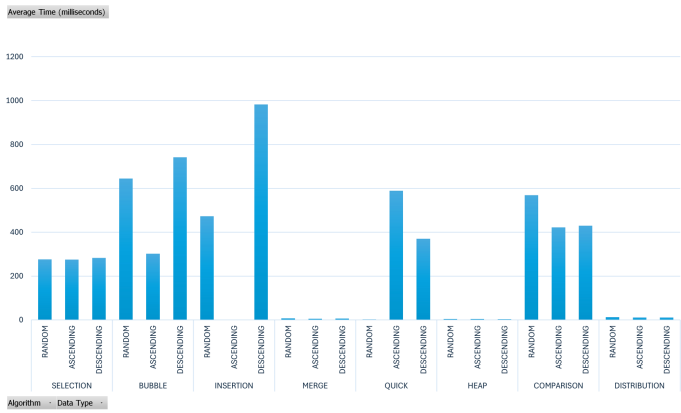


Fig. 3: chart showing results for randomized, ascending, and descending average execution time for all algorithms for 10,000 input data points

จาก (2) เมื่อขนาดข้อมูลเพิ่มขึ้นเป็น 5,000 ตัว Merge Sort และ Heap Sort ยังคงรักษาประสิทธิภาพที่ดีไว้ได้ โดยใช้เวลาเพียง 2-4 มิลลิวินาทีในการเรียงข้อมูล Insertion Sort ยังคงทำงานได้ดีเยี่ยมกับข้อมูลที่เรียงจากน้อยไปมาก แต่ใช้เวลามากถึง 388 มิลลิวินาทีสำหรับข้อมูลที่เรียงจากมากไปน้อย

Selection Sort และ Bubble Sort แสดงให้เห็นข้อจำกัดด้านประสิทธิภาพอย่างชัดเจน โดยใช้เวลา 70-77 มิลลิวินาทีสำหรับ Selection Sort และ 77-183 มิลลิวินาทีสำหรับ Bubble Sort สำหรับ Quick Sort เริ่มแสดงปัญหาเมื่อต้องจัดการกับข้อมูลที่เรียงลำดับแล้ว โดยใช้เวลา 95-160 มิลลิวินาทีในกรณีดังกล่าว Comparison-Counting Sort ใช้เวลาสำหรับชุดข้อมูลสุ่มมากกว่าชุดข้อมูลเรียงเล็กน้อย และ Distribution-Counting Sort ยังคงแสดงพฤติกรรมที่คงที่โดยใช้เวลาประมาณ 10 มิลลิวินาทีไม่ว่าข้อมูลจะอยู่ในรูปแบบใด

จาก (3) ที่ขนาดข้อมูล 10,000 ตัว ความแตกต่างระหว่างอัลกอริทึมยิ่งเห็นได้ชัดเจนขึ้น Merge Sort และ Heap Sort ยังคงมีประสิทธิภาพสูงสุด โดยใช้เวลาเพียง 3-7 มิลลิวินาที Insertion Sort ยังคงทำงานได้ดีเยี่ยมกับข้อมูลที่เรียงจากน้อยไปมาก แต่ใช้เวลาเพิ่มขึ้นเป็น 982 มิลลิวินาทีสำหรับข้อมูลที่เรียงจากมากไปน้อย

Selection Sort และ Bubble Sort ยังคงมีข้อจำกัดด้านประสิทธิภาพอย่างชัดเจน โดยใช้เวลา 275-282 มิลลิวินาทีสำหรับ Selection Sort และ 302-741 มิลลิวินาทีสำหรับ Bubble Sort สำหรับ Quick Sort มีปัญหาอย่างมากกับข้อมูลที่เรียงลำดับแล้ว โดยใช้เวลา 369-589 มิลลิวินาที แต่ทำงานได้ดีมากกับข้อมูลสุ่ม (2.2 มิลลิวินาที) Comparison-Counting Sort ยังคงใช้เวลาสำหรับชุดข้อมูลสุ่มมากกว่าชุดข้อมูลเรียงเล็กน้อย และ Distribution-Counting Sort ยังคงรักษาประสิทธิภาพที่คงที่ โดยใช้เวลาประมาณ 10-12 มิลลิวินาที

D. ชุดข้อมูล 25,000 ตัว

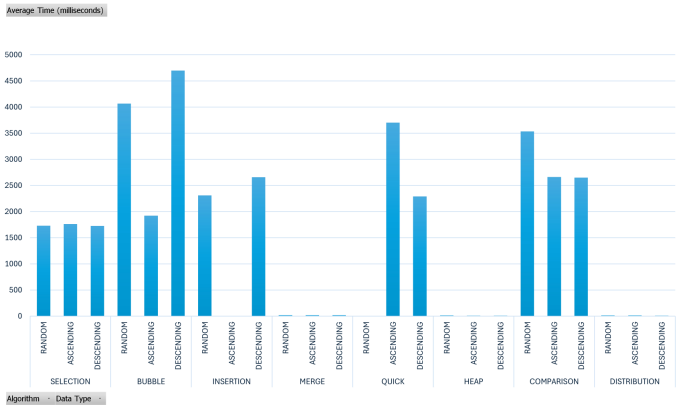


Fig. 4: chart showing results for randomized, ascending, and descending average execution time for all algorithms for 25,000 input data points

(4) ที่ขนาดข้อมูล 25,000 ตัว ความแตกต่างด้านประสิทธิภาพระหว่างอัลกอริทึมเห็นได้ชัดเจนที่สุด Merge Sort และ Heap Sort ยังคงรักษาประสิทธิภาพที่ดีเยี่ยม โดยใช้เวลาเพียง 10-19 มิลลิวินาที Insertion Sort ยังคงทำงานได้ดีที่สุดกับข้อมูลที่เรียงจากน้อยไปมาก (0.31 มิลลิวินาที) แต่ใช้เวลามากถึง 2,656 มิลลิวินาทีสำหรับข้อมูลที่เรียงจากมากไปน้อย

Selection Sort และ Bubble Sort แสดงให้เห็นข้อจำกัดอย่างชัดเจน โดยใช้เวลา 1,727-1,760 มิลลิวินาทีสำหรับ Selection Sort และ 1,919-4,699 มิลลิวินาทีสำหรับ Bubble Sort สำหรับ Quick Sort ยังคงมีปัญหาอย่างมากกับข้อมูลที่เรียงลำดับแล้ว โดยใช้เวลา 2,290-3,700 มิลลิวินาที แต่ยังคงทำงานได้ดีมากกับข้อมูลสุ่ม (5.43 มิลลิวินาที)

สำหรับ Comparison-Counting Sort ยังคงใช้เวลาสำหรับชุดข้อมูลสุ่มมากกว่าชุดข้อมูลเรียงเล็กน้อย และ Distribution-Counting Sort ยังคงรักษาประสิทธิภาพที่คงที่ โดยใช้เวลาประมาณ 11-12 มิลลิวินาที

## V. สรุปผล

### A. Worst-Case และ Best-Case

1) Selection Sort: แสดงให้เห็นว่ามีพฤติกรรมค่อนข้างคงที่ไม่ว่าข้อมูลจะอยู่ในรูปแบบใด โดยเวลาที่ใช้ในการเรียงข้อมูลแต่ละรูปแบบ (สุ่ม, เรียงจากน้อยไปมาก, เรียงจากมากไปน้อย) มีความแตกต่างกันเพียงเล็กน้อย

2) Bubble Sort: แสดงความแตกต่างของประสิทธิภาพอย่างชัดเจนระหว่างข้อมูลที่เรียงลำดับแล้วและยังไม่เรียงลำดับ กรณีที่แย่ที่สุดคือเมื่อข้อมูลเรียงจากมากไปน้อย ในขณะที่กรณีที่ดีที่สุดคือเมื่อข้อมูลเรียงจากน้อยไปมากอยู่แล้ว แสดงให้เห็นว่า Bubble Sort มีประสิทธิภาพดีกับข้อมูลที่เกือบเรียงลำดับอยู่แล้ว

3) Insertion Sort: แสดงความแตกต่างของประสิทธิภาพอย่างเห็นได้ชัดที่สุดจากทุกอัลกอริทึม โดยกรณีที่ดีที่สุดคือเมื่อข้อมูลเรียงจากน้อยไปมากอยู่แล้ว ใช้เวลาเพียง 0-0.4 มิลลิวินาที ไม่ว่าขนาดข้อมูลจะเป็นเท่าใด ในทางตรงกันข้าม กรณีที่แย่ที่สุดคือเมื่อข้อมูลเรียงจากมากไปน้อย

4) Merge Sort: แสดงให้เห็นประสิทธิภาพที่คงที่ในทุกกรณี โดยมีความแตกต่างของเวลาที่ใช้ในแต่ละรูปแบบข้อมูลไม่มากนัก กรณีที่แย่ที่สุดคือข้อมูลสุ่ม และกรณีที่ดีที่สุดคือข้อมูลเรียงน้อยไปมากอยู่แล้ว แสดงให้เห็นว่า Merge Sort มีประสิทธิภาพที่ดีและเสถียรในทุกสถานการณ์

5) Quick Sort: แสดงความแตกต่างของประสิทธิภาพอย่างมากระหว่างข้อมูลสุ่มและข้อมูลที่เรียงลำดับแล้ว โดยกรณีที่แย่ที่สุดคือเมื่อข้อมูลเรียงจากน้อยไปมาก ในขณะที่กรณีที่ดีที่สุดคือข้อมูลสุ่ม แสดงให้เห็นว่า Quick Sort มีประสิทธิภาพดีมากกับข้อมูลสุ่มแต่ไม่เหมาะกับข้อมูลที่เรียงลำดับแล้ว

6) Heap Sort: แสดงประสิทธิภาพที่ดีเยี่ยมและคงที่ในทุกรูปแบบข้อมูล แสดงให้เห็นว่า Heap Sort มีประสิทธิภาพที่ดีและเสถียรไม่ว่าข้อมูลจะอยู่ในรูปแบบใด

7) Comparison-Counting Sort: แสดงให้เห็นประสิทธิภาพที่ดีกว่าเล็กน้อยในกรณีข้อมูลเรียงลำดับ โดยกรณีที่แย่ที่สุดคือข้อมูลสุ่ม แสดงให้เห็นว่าประสิทธิภาพจะดีขึ้นเล็กน้อยเมื่อข้อมูลมีการเรียงลำดับ

8) Distribution-Counting Sort: แสดงลักษณะพิเศษที่แตกต่างจากอัลกอริทึมอื่นๆ โดยใช้เวลาที่ค่อนข้างคงที่ไม่ว่าข้อมูลจะมีขนาดหรือรูปแบบใด แสดงให้เห็นว่า Distribution Sort มีประสิทธิภาพที่คงที่และไม่ขึ้นกับรูปแบบของข้อมูล

### B. Algorithm ที่ดีที่สุด

จากผลการทดสอบ สามารถระบุอัลกอริทึมที่มีประสิทธิภาพสูงสุดในแต่ละกรณี สำหรับข้อมูลที่เรียงจากน้อยไปมากอยู่แล้ว Insertion Sort เป็นตัวเลือกที่ดีที่สุดโดยใช้เวลาเพียง 0-0.4 มิลลิวินาทีแม้กับข้อมูลขนาดใหญ่ถึง 25,000 รายการ สำหรับข้อมูลสุ่ม Quick Sort ทำงานได้ดีเยี่ยมโดยใช้เวลาเพียง 2-5 มิลลิวินาทีแม้กับข้อมูลขนาดใหญ่ ในขณะที่ Merge Sort และ Heap Sort แสดงประสิทธิภาพที่ดีและคงที่ในทุกรูปแบบข้อมูล โดยใช้

เวลาไม่เกิน 20 มิลลิวินาทีแม้กับข้อมูลขนาด 25,000 รายการ Distribution Sort แสดงประสิทธิภาพที่น่าสนใจด้วยเวลาที่คงที่ประมาณ 10-12 มิลลิวินาทีไม่ว่าข้อมูลจะมีขนาดหรือรูปแบบใด

หากพิจารณาการนำไปใช้งานจริง Insertion Sort เหมาะสำหรับข้อมูลขนาดเล็กหรือข้อมูลที่เกือบเรียงลำดับอยู่แล้ว Quick Sort เหมาะสำหรับข้อมูลสุ่มทั่วไปที่ไม่มีการเรียงลำดับมาก่อน Merge Sort และ Heap Sort เหมาะสำหรับการทำงานที่ต้องการความเสถียรและคาดการณ์เวลาการทำงานได้ โดยไม่ขึ้นกับลักษณะข้อมูล

## References


[1] Levitin, A. (2012). Introduction to the design & analysis of algorithms (3. ed., international ed). Pearson.

[2] Expert-Programming-Tutor.com. (ม.ป.ป.-b). Selection Sort in C | EPT | Expert-Programming-Tutor. Expert-Programming-Tutor.Com. สืบค้น 26 พฤศจิกายน 2024, จาก https:// expert-programming-tutor.com/ tutorial/ article/ KC0010062002_Selection_Sort_in_C.php

[3] Chen, R. (2017, ธันวาคม 4). Selection sort loop questions in Java [Forum post]. Stack Overflow. https://stackoverflow.com/q/47622765

[4] Bubble Sort คือ? | Matter Devs. (2023, กุมภาพันธ์ 16). https://www.matterdevs.com/bubble-sort/

[5] Solved he pseudocode for bubble sort is as follows: | Chegg.com. (ม.ป.ป.). สืบค้น 26 พฤศจิกายน 2024, จาก https://www.chegg.com/ homework-help/ questions-and-answers/ pseudocode-bubble-sort-follows-procedure-bubblesort-list-sortable-items-n-length-repeat-sw-q116414771

[6] Expert-Programming-Tutor.com. (ม.ป.ป.-a). การเรียงลำดับแบบต่าง ๆ | EPT | Expert-Programming-Tutor. Expert-Programming-Tutor.Com. สืบค้น 26 พฤศจิกายน 2024, จาก https:// expert-programming-tutor.com/tutorial/data_structure/01_Sorting2.php

[7] Merge Sort —Data Structure and Algorithms Tutorials. (2013, มีนาคม 15). GeeksforGeeks. https:// www.geeksforgeeks.org/ merge-sort/

[8] fResult – Medium. (ม.ป.ป.). สืบค้น 26 พฤศจิกายน 2024, จาก https://fresult.medium.com/

[9] Pontawan Jansuk: Heap Sort. (ม.ป.ป.). Pontawan Jansuk. สืบค้น 26 พฤศจิกายน 2024, จาก https://pontawan.blogspot.com/p/heap-sort.html

[10] Solved Heap Sort Here is the pseudocode for Heap Sort, | Chegg.com. (ม.ป.ป.). สืบค้น 26 พฤศจิกายน 2024, จาก https://www.chegg.com/ homework-help/ questions-and-answers/ heap-sort-pseudocode-heap-sort-modified-include-counter-count-0-heapsort-1-2-buildmaxheap–q24837890question-transcript